

“学”：基于 Nexys A7 的 VGA 游戏集合机

从 VGA 显示、PS/2 输入到资源分析与时序收敛

Nexys A7-100T

VGA / PS2 / Game Slot

LUT / WNS / Bitstream

这不是单个小游戏，而是一台 FPGA 游戏集合机

它有完整用户流程：菜单、选择、启动、返回。

上电进入 VGA 菜单

W/S 或方向键选择

Enter/Space 启动

Esc 从游戏返回菜单

- ☐ 上电进入 VGA 菜单
- ☐ W/S 或方向键选择
- ☐ Enter/Space 启动
- ☐ Esc 从游戏返回菜单

本次汇报主线：从能显示，到能优化

按学习路线讲清系统能力，而不是按文件列表堆代码。

VGA 实时扫描

菜单像素渲染

PS/2 协议接收

Game Slot 平台化

资源分析

post-route 时序收敛

- ☐ VGA 实时扫描
- ☐ 菜单像素渲染
- ☐ PS/2 协议接收
- ☐ Game Slot 平台化

系统总体架构：公共服务 + 游戏槽位

`game\_console\_top` 像主机，slot1~slot4 像卡带。

公共服务 + 游戏槽位
统一输出仲裁

- ☐ 公共服务：VGA timing、PS/2 byte、菜单控制、菜单渲染
- ☐ 槽位服务：游戏状态、游戏渲染、外设反馈
- ☐ 顶层仲裁：VGA / LED / 七段管 / buzzer

VGA 的本质：扫描到哪里，就输出哪里的 RGB

显示器不接收整张图片，FPGA 必须每个像素周期实时给颜色。



640×480 可见区
扫描路径 + 当前像素

- ☐ 640x480 是可见区，消隐区还负责同步
- ☐ ``pixel_x/y`` 是当前扫描坐标
- ☐ ``display_active`` 区分可见区和消隐区
- ☐ RGB 与 HS/VS 同步输出

VGA 时序链路：从 100MHz 到 RGB 输出

`pixel\_tick` 让 VGA 坐标按像素节拍推进，渲染器按坐标出颜色。

100MHz 主时钟不变 `pix\_div` 产生四拍一次的像素使能 `h\_count/v\_count` 形成扫描坐标 `renderer` 在当前坐标上实时决定 RGB

- ☐ 100MHz 主时钟不变
- ☐ `pix\_div` 产生四拍一次的像素使能
- ☐ `h\_count/v\_count` 形成扫描坐标
- ☐ `renderer` 在当前坐标上实时决定 RGB

`console\_vga\_sync`: 两个计数器生成 VGA 坐标

参数、像素使能、坐标和可见区判断构成 VGA 控制器核心。

```
13 localparam integer H_VISIBLE = 640;
19 localparam integer H_FRONT  = 16;
20 localparam integer H_SYNC   = 96;
21 localparam integer H_BACK   = 48;
25 localparam integer V_VISIBLE = 480;
31 localparam integer V_FRONT  = 10;
32 localparam integer V_SYNC   = 2;
33 localparam integer V_BACK   = 33;

43 reg [1:0] pix_div;
44 assign pixel_tick = (pix_div == 2'b11);
45 assign display_active =
46     (h_count < H_VISIBLE) && (v_count < V_VISIBLE);
47 else if (pixel_tick) begin
48     if (h_count == H_TOTAL - 1) ...
```

`H\_VISIBLE=640`、`V\_VISIBLE=480` 定义可见区

`pixel\_tick` 每四个主时钟有效一次

`h\_count/v\_count` 只在 `pixel\_tick` 下推进

`display\_active` 控制消隐区 RGB 输出

为什么用 pixel_tick，而不是新建 25MHz 时钟

用 clock enable 保持单时钟域，降低跨时钟域和约束复杂度。

不引入额外时钟域 VGA 坐标只在 `pixel\_tick` 游戏逻辑和公共服务仍由 `CLK100MHZ` 驱动 后续 multicycle 只用于明确的 pixel path

- ☐ 不引入额外时钟域
- ☐ VGA 坐标只在 `pixel\_tick` 推进
- ☐ 游戏逻辑和公共服务仍由 `CLK100MHZ` 驱动
- ☐ 后续 multicycle 只用于明确的 pixel path

菜单不是图片，而是像素级坐标判断

标题、菜单项、高亮和边框都由当前像素命中关系决定。

`pixel\_x/y` 进入区域判断

`cursor` 决定哪一项高亮

字模和文字由 case/function 展开

菜单渲染也是 LUT 压力来源

- ☐ `pixel\_x/y` 进入区域判断
- ☐ `cursor` 决定哪一项高亮
- ☐ 字模和文字由 case/function 展开
- ☐ 菜单渲染也是 LUT 压力来源

菜单渲染逻辑：区域、字模、颜色优先级

渲染器只回答“这个像素怎么画”，控制器只回答“当前选谁”。

```
// src/common/console_menu_renderer.v ????  
if (!display_active)  
    rgb = 12'h000;           // ????  
else begin  
    rgb = background_color; // ????  
    if (title_on)    rgb = title_color;  
    else if (help_on) rgb = help_color;  
    else if (item_text_on)  
        rgb = selected ? highlight_text : normal_text;  
    else if (item_body)  
        rgb = selected ? highlight_box : item_box;  
end
```

消隐区输出黑色

标题、帮助文字、菜单项分别判断

选中项改变文字和边框颜色

最终按优先级覆盖 RGB

菜单控制与输出复用：状态决定显示源

`menu\_active` 把菜单 RGB 和当前 slot RGB 复用到同一组 VGA 端口。



- ☐ PS/2 byte 转成 cursor / game_sel
- ☐ 菜单激活时显示菜单
- ☐ 游戏运行时显示 active slot
- ☐ HS/VS 始终来自公共 VGA 同步器

PS/2 输入不是按键电平，而是 11-bit 串行帧

键盘发的是扫描码，FPGA 先接收 byte，再解释成事件。

start → D0~D7 → parity → stop
下降沿采样

- ☐ start bit 为 0
- ☐ 8-bit scan code, LSB first
- ☐ parity 用于校验
- ☐ stop bit 为 1

`console\_ps2\_rx`: 同步、滤波、下降沿采样

接收器只输出 `byte\_ready + byte\_data`, 不直接判断 W/S 或方向键。

```
40 assign ps2_clk_fall_next = ps2_clk_q & ~ps2_clk_next;
41 assign ps2_parity_ok = (^shift_data) ^ parity_bit;

70 if (ps2_clk_fall_next) begin
72     case (bit_count)
73         4'd0: if (ps2_data_next == 1'b0)
74             bit_count <= 4'd1;
75         4'd1: shift_data[0] <= ps2_data_next;
76         ...
111        4'd9: parity_bit <= ps2_data_next;
115        4'd10: if (ps2_data_next == 1'b1 &&
116                    ps2_parity_ok) begin
117            byte_data <= shift_data;
118            byte_ready <= 1'b1;
```

PS/2 clock/data 先同步并滤波

`ps2\_clk\_fall\_next` 检测下降沿

`bit\_count` 管理帧位置

stop 和 parity 通过后产生 `byte\_ready`

扫描码事件化：从 byte 到菜单动作

W/S、方向键、Enter/Space、Esc 被转换成系统事件。

`F0`：释放前缀，避免松键触发

`E0`：扩展键前缀

菜单中 Enter/Space 启动

游戏中 Esc 返回菜单

- ☐ `F0`：释放前缀，避免松键触发
- ☐ `E0`：扩展键前缀
- ☐ 菜单中 Enter/Space 启动
- ☐ 游戏中 Esc 返回菜单

Game Slot API：从“多个游戏”到“游戏平台”

每个游戏像卡带一样接入，公共输入和统一输出保持稳定。



game_console_top
主机 + 卡带式 slot

- ☐ 公共输入：clk、reset、selected
- ☐ 显示输入：frame_tick、pixel_tick、display_active、pixel_x/y
- ☐ 键盘事件：ps2_byte_ready、ps2_byte_data
- ☐ 统一输出：RGB、LED、七段管、buzzer

输出仲裁：一组真实外设，多个画面来源

菜单和四个游戏共享 VGA、LED、七段管和蜂鸣器。



- ☐ slot1~slot4 先复用为 active slot
- ☐ 菜单态覆盖 active slot 输出
- ☐ 未选游戏不等于综合时不占资源
- ☐ 裁剪资源需要 stub 或构建宏

资源报告：LUT 高、FF 低，问题是组合逻辑太重

30146 LUT 约 48%，4371 FF 约 3%，优化方向是切短组合路径。

LUT 30146 / 63400 $\approx$ 48%

FF 4371 / 126800 $\approx$ 3%

LUT 高 + FF 低
组合逻辑太重

像素渲染

tile 判断

字体/菜单

sprite 命中

图层优先级

case/if/function

为什么 VGA 游戏容易 LUT 高

每个像素都要判断背景、地图、对象、文字和图层优先级。

像素渲染坐标比较

tile/map 查询

字体和菜单字模

sprite 命中检测

对象图层优先级

case / if / function 展开

- ☐ 像素渲染坐标比较
- ☐ tile/map 查询
- ☐ 字体和菜单字模
- ☐ sprite 命中检测

时序问题：代码能综合，不代表能按 100MHz 跑

WNS 为负说明至少一条寄存器到寄存器路径来不及。

100MHz 周期为 10ns

组合逻辑必须在一个周期内稳定

WNS: 最差路径余量

TNS: 所有失败路径负 slack 总和

- ☐ 100MHz 周期为 10ns
- ☐ 组合逻辑必须在一个周期内稳定
- ☐ WNS: 最差路径余量
- ☐ TNS: 所有失败路径负 slack 总和

从真实负 slack 到 post-route 收敛

先去掉宽泛例外暴露真实基线，再通过 RTL staging 把 WNS 拉正。

Before
WNS=-0.445ns
TNS=-18.104ns
Fail=94

After
WNS=+0.174ns
TNS=0
Fail=0

真实基线

Slot2 staging

Slot3 try_x/y

Slot4 probe/eval/apply

VGA multicycle

post-route

三个优化案例：把一拍大计算拆成多拍小计算

用户可见行为不变，但每拍组合逻辑变短。

Slot2: ghost/lock 拆多拍

用户行为不变

Slot3: try_x/try_y 寄存

组合逻辑变短

Slot4: TEST/EVAL/APPLY

WNS 转正

学习收获：从功能实现到工程实现

真正的 FPGA 产品要同时满足显示、输入、架构、资源和时序。

VGA：实时像素扫描

PS/2：协议解析和事件化

Game Slot：平台化接口

LUT/FF：资源瓶颈判断

WNS/TNS：post-route 工程收敛

- ☐ VGA：实时像素扫描
- ☐ PS/2：协议解析和事件化
- ☐ Game Slot：平台化接口
- ☐ LUT/FF：资源瓶颈判断